

Problem Statement: Number Utility Application

Develop a Java program to create a `NumberUtility` class that provides various mathematical operations on an integer using static methods.

The application should accept an integer from the user and perform the following operations:

1. **Check Even/Odd** – Determine whether the given number is even or odd.
2. **Check Prime** – Determine whether the given number is a prime number.
3. **Calculate Factorial** – Calculate the factorial of the given number.
4. **Reverse Number** – Reverse the digits of the given number.
5. **Check Palindrome** – Determine whether the given number is a palindrome.
6. **Find Sum of Digits** – Calculate the sum of all digits of the given number.

Functional Requirements

Create a class named `NumberUtility` containing the following static methods:

```
isEven(int number)
isPrime(int number)
factorial(int number)
reverse(int number)
isPalindrome(int number)
sumOfDigits(int number)
```

Each method should perform its respective operation and return an appropriate result.

Create a separate `NumberUtilityDemo` class containing the `main()` method.

The `main()` method should:

- Accept an integer from the user.
- Invoke all methods using the **class name**.
- Display the results in a properly formatted manner.
- **Do not create an object of the `NumberUtility` class.**

Static methods are class methods and can be invoked using the class name without creating an instance, which is the key concept this exercise is intended to demonstrate. ([Oracle Docs](#))

Sample Input

Enter a number: 121

Expected Output

```
===== NUMBER UTILITY =====  
  
Number           : 121  
Even/Odd         : Odd  
Prime            : Not Prime  
Factorial        : 51090942171709440000  
Reverse          : 121  
Palindrome       : Yes  
Sum of Digits    : 4
```

Note: For the factorial operation, students should consider an appropriate data type and input range to avoid integer overflow.

Important Constraints

- All utility operations must be implemented as **static methods**.
- Methods should accept the required input through parameters.
- Methods should return results rather than directly handling all output.

- The `main()` method should be responsible for displaying the results.
- No object of `NumberUtility` should be created.
- Appropriate validation should be performed for invalid inputs, particularly for factorial.

Learning Objectives

After completing this problem, students should be able to:

- Define and use **static methods**.
- Invoke static methods using `ClassName.methodName()`.
- Pass arguments to static methods.
- Return values from methods.
- Break a program into reusable methods.
- Understand the difference between **class methods and instance methods**.
- Apply loops and conditional statements to solve numerical problems.
- Design a simple utility class following separation of responsibilities.

Optional Extension

To make the problem more suitable for an **advanced lab**, add a menu-driven interface:

```
===== NUMBER UTILITY =====
1. Check Even/Odd
2. Check Prime
3. Calculate Factorial
4. Reverse Number
5. Check Palindrome
6. Sum of Digits
7. Exit

Enter your choice:
```

This extension allows students to demonstrate **static methods + method parameters + return values + loops + conditional statements + switch-case + user input** in one application.